PROYECTO DE DESARROLLO DE SERVICIOS WEB & CLOUD

BASES DE DATOS NO RELACIONALES • ARQUITECTURA REST

NODE.JS • EXPRESS • MONGODB ATLAS • VERCEL

DOCUMENTACIÓN Y REPORTE TÉCNICO: SERVICIO API REST PARA COLECCIÓN "EMPLEADOS"

Adaptación, implementación completa de operaciones CRUD (POST, GET, PUT, DELETE), persistencia en clúster NoSQL MongoDB Atlas y despliegue serverless en la nube con Vercel.

AUTOR / DESARROLLADOR

Jesús Zamora

BASE DE DATOS

MongoDB Atlas (No-Relational DB)

PLATAFORMA DE HOSTING

Vercel Cloud (Serverless Architecture)

FECHA DE ENTREGA

Octubre 2026

1. Información General del Servicio y Hosting

El presente proyecto expone un servicio API REST desarrollado con **Node.js** y **Express**, integrado mediante el ODM **Mongoose** hacia una base de datos No Relacional alojada en **MongoDB Atlas**, y publicado en producción de manera continua a través de **Vercel** como funciones Serverless de alta disponibilidad.

Proveedor de Hosting:	**Vercel** (Vercel Serverless Functions)
URL Base de Producción:	`https://miapi-pi.vercel.app`
URL Endpoint Principal:	`https://miapi-pi.vercel.app/api/empleados`
Repositorio GitHub:	`https://github.com/Jesuszruelas/webAPI`
Herramienta de Prueba:	**Postman** (REST Client / Automated Runner)
Base de Datos:	MongoDB Atlas Cluster (Base de datos: `miBD`)
Nombre de la Colección:	`Empleados` (Colección No Relacional)

2. Especificación del Modelo de Datos ("Empleados")

La colección **"Empleados"** fue adaptada rigurosamente a las especificaciones requeridas, garantizando tipos de datos fuertemente validados y almacenamiento nativo de fechas en formato **BSON Date**:

CAMPO	TIPO DE DATO	TIPO EN MONGO/BSON	OBLIGATORIO	DESCRIPCIÓN Y REGLAS
Clave	Numérica	`Number / Double`	Sí (Única)	Identificador numérico principal único del empleado.
Nombre	Texto	`String`	Sí	Nombre completo del empleado (soporta espacios y acentos).
dirección	Texto	`String`	Sí	Domicilio o dirección residencial del empleado.
fechadenacimiento	BSON Date	`Date (ISODate)`	Sí	Fecha de nacimiento almacenada nativamente como BSON Date.
sexo	Numérico o Texto	`Mixed (String/Number)`	Sí	Acepta texto ("M", "F", "Masculino") o código numérico (1, 2).
tipodesangre	Texto	`String`	Sí	Grupo sanguíneo y factor Rh (ej: "O+", "A+", "B-").

> **Nota de Conexión en MongoDB Atlas:** Para que Vercel (o cualquier cliente en la nube) pueda comunicarse con el clúster Atlas, debe estar habilitada la regla de red `0.0.0.0/0` (Allow Access from Anywhere) en *Network Access* de MongoDB Atlas.

3. Ligas y URLs para Pruebas en el Hosting (En la Nube)

A continuación se detallan las rutas del servicio publicado en la nube para ser ejecutadas en Postman:

MÉTODO	ENDPOINT EN LA NUBE (VERCEL)	DESCRIPCIÓN DE LA OPERACIÓN
GET	`https://miapi-pi.vercel.app/`	Verifica el estado del servicio y lista la documentación interactiva.
GET	`https://miapi-pi.vercel.app/api/empleados`	Consulta y retorna todos los documentos de la colección *Empleados*.
GET	`https://miapi-pi.vercel.app/api/empleados/:clave`	Consulta un empleado específico por su *Clave* numérica o por su *_id*.
POST	`https://miapi-pi.vercel.app/api/empleados`	Crea e inserta un nuevo empleado en MongoDB Atlas con validación de datos.
PUT	`https://miapi-pi.vercel.app/api/empleados/:clave`	Actualiza los campos de un empleado existente identificado por su *Clave*.
DELETE	`https://miapi-pi.vercel.app/api/empleados/:clave`	Elimina permanentemente el empleado especificado por su *Clave*.

4. Ejemplos de RAW JSON para Pruebas en Postman

A) Operación POST — Crear Empleado

Configuración en Postman: Seleccionar método `POST`, URL `https://miapi-pi.vercel.app/api/empleados`, pestaña *Body* > *raw* > formato *JSON*.

PAYLOAD RAW JSON DE ENTRADA:

```
{
  "Clave": 101,
  "Nombre": "Carlos Mendoza López",
  "dirección": "Av. Revolución #450, Col. Centro",
  "fechadenacimiento": "1992-05-14",
  "sexo": "M",
  "tipodesangre": "O+"
}
```

RESPUESTA ESPERADA DE LA API (HTTP 201 CREATED):

```
{
  "mensaje": "Empleado registrado exitosamente",
  "empleado": {
    "Clave": 101,
    "Nombre": "Carlos Mendoza López",
```

```
    "dirección": "Av. Revolución #450, Col. Centro",
    "fechadenacimiento": "1992-05-14T00:00:00.000Z",
    "sexo": "M",
    "tipodesangre": "O+",
    "_id": "67b9d4e5f123456789abcdef",
    "createdAt": "2026-10-22T20:25:00.000Z",
    "updatedAt": "2026-10-22T20:25:00.000Z"
  }
}
```

B) Operación GET — Listar Todos los Empleados

Configuración en Postman: Método `GET` hacia `https://miapi-pi.vercel.app/api/empleados`.

RESPUESTA ESPERADA DE LA API (HTTP 200 OK):

```
{
  "total": 1,
  "datos": [
    {
      "_id": "67b9d4e5f123456789abcdef",
      "Clave": 101,
      "Nombre": "Carlos Mendoza López",
      "dirección": "Av. Revolución #450, Col. Centro",
      "fechadenacimiento": "1992-05-14T00:00:00.000Z",
      "sexo": "M",
      "tipodesangre": "O+",
      "createdAt": "2026-10-22T20:25:00.000Z",
      "updatedAt": "2026-10-22T20:25:00.000Z"
    }
  ]
}
```

C) Operación GET — Consultar Empleado por Clave

Configuración en Postman: Método `GET` hacia `https://miapi-pi.vercel.app/api/empleados/101`.

RESPUESTA ESPERADA DE LA API (HTTP 200 OK):

```
{
  "_id": "67b9d4e5f123456789abcdef",
  "Clave": 101,
  "Nombre": "Carlos Mendoza López",
  "dirección": "Av. Revolución #450, Col. Centro",
  "fechadenacimiento": "1992-05-14T00:00:00.000Z",
  "sexo": "M",
  "tipodesangre": "O+"
}
```

D) Operación PUT — Actualizar Empleado

Configuración en Postman: Método `PUT` hacia `https://miapi-pi.vercel.app/api/empleados/101`, pestaña *Body > raw > JSON*.

PAYLOAD RAW JSON DE ENTRADA:

```
{
  "Nombre": "Carlos Mendoza López (Actualizado)",
  "dirección": "Blvd. Paseo de los Héroes #1200, Zona Río",
  "tipodesangre": "O-"
}
```

RESPUESTA ESPERADA DE LA API (HTTP 200 OK):

```
{
  "mensaje": "Empleado actualizado exitosamente",
  "empleado": {
    "_id": "67b9d4e5f123456789abcdef",
    "Clave": 101,
    "Nombre": "Carlos Mendoza López (Actualizado)",
    "dirección": "Blvd. Paseo de los Héroes #1200, Zona Río",
    "fechadenacimiento": "1992-05-14T00:00:00.000Z",
    "sexo": "M",
    "tipodesangre": "O-"
  }
}
```


E) Operación DELETE — Eliminar Empleado

Configuración en Postman: Método `DELETE` hacia `https://miapi-pi.vercel.app/api/empleados/101`.

RESPUESTA ESPERADA DE LA API (HTTP 200 OK):


```
{
  "mensaje": "Empleado eliminado exitosamente de la colección Empleados",
  "empleado": {
    "_id": "67b9d4e5f123456789abcdef",
    "Clave": 101,
    "Nombre": "Carlos Mendoza López (Actualizado)"
  }
}
```

5. Código Fuente Completo del Servicio

A continuación se presenta el código fuente modularizado y adaptado para arquitectura serverless en Vercel:

ARCHIVO: INDEX.JS (LÓGICA PRINCIPAL DEL SERVICIO EXPRESS & MONGOOSE)

```
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');

const app = express();

// Middlewares globales
app.use(cors());
app.use(express.json());

// Conexión reutilizable a MongoDB Atlas (Serverless-friendly)
let cachedDb = null;

async function connectToDatabase() {
    if (cachedDb && mongoose.connection.readyState === 1) {
        return cachedDb;
    }
    const mongoUri = process.env.MONGO_URI;
    if (!mongoUri) {
        throw new Error('La variable de entorno MONGO_URI no está configurada.');
    }
    cachedDb = await mongoose.connect(mongoUri, { serverSelectionTimeoutMS: 5000 });
    return cachedDb;
}

// Esquema y Modelo para la colección "Empleados"
const empleadoSchema = new mongoose.Schema({
    Clave: {
        type: Number,
        required: [true, 'El campo Clave (numérica) es obligatorio'],
        unique: true
    },
    Nombre: {
        type: String,
        required: [true, 'El campo Nombre (texto) es obligatorio'],
        trim: true
    },
    dirección: {
        type: String,
        required: [true, 'El campo dirección (texto) es obligatorio'],
        trim: true,
        alias: 'direccion'
    },
    fechadenacimiento: {
        type: Date,
        required: [true, 'El campo fechadenacimiento (BSON Date) es obligatorio']
    },
```

```
    sexo: {
        type: mongoose.Schema.Types.Mixed,
        required: [true, 'El campo sexo (numérico o texto) es obligatorio']
    },
    tipodesangre: {
        type: String,
        required: [true, 'El campo tipodesangre (texto) es obligatorio'],
        trim: true
    }
}, { timestamps: true, versionKey: false });

const Empleado = mongoose.models.Empleado || mongoose.model('Empleado', empleadoSchema, 'Empleados');

// Helper de normalización
function normalizarDatosEmpleado(data) {
    const n = {};
    if (data.Clave !== undefined || data.clave !== undefined) n.Clave = Number(data.Clave ?? data.clave);
    if (data.Nombre !== undefined || data.nombre !== undefined) n.Nombre = String(data.Nombre ??
data.nombre).trim();
    if (data.dirección !== undefined || data.direccion !== undefined) n.dirección = String(data.dirección
?? data.direccion).trim();
    if (data.fechadenacimiento || data.fechaDeNacimiento) n.fechadenacimiento = new
Date(data.fechadenacimiento || data.fechaDeNacimiento);
    if (data.sexo !== undefined || data.Sexo !== undefined) n.sexo = data.sexo ?? data.Sexo;
    if (data.tipodesangre || data.tipoDeSangre) n.tipodesangre = String(data.tipodesangre ||
data.tipoDeSangre).trim();
    return n;
}

// Middleware de verificación de BD
const requireDB = async (req, res, next) => {
    try {
        await connectToDatabase();
        next();
    } catch (error) {
        res.status(500).json({ error: 'Error al conectar con MongoDB Atlas', detalle: error.message });
    }
};
```

ARCHIVO: INDEX.JS (CONTINUACIÓN: ENDPOINTS REST)

```
// GET: Listar todos los empleados
app.get('/api/empleados', requireDB, async (req, res) => {
    try {
        const empleados = await Empleado.find().sort({ Clave: 1 });
        res.status(200).json({ total: empleados.length, datos: empleados });
    } catch (error) {
        res.status(500).json({ error: 'Error al consultar empleados', mensaje: error.message });
    }
});

// GET: Obtener un empleado por Clave o _id
app.get('/api/empleados/:identificador', requireDB, async (req, res) => {
    try {
        const { identificador } = req.params;
        const filtro = !isNaN(identificador) ? { Clave: Number(identificador) } : { _id: identificador };
        const empleado = await Empleado.findOne(filtro);
        if (!empleado) return res.status(404).json({ error: 'Empleado no encontrado' });
        res.status(200).json(empleado);
    } catch (error) {
        res.status(500).json({ error: error.message });
    }
});

// POST: Registrar un nuevo empleado
app.post('/api/empleados', requireDB, async (req, res) => {
    try {
        const datos = normalizarDatosEmpleado(req.body);
        const existente = await Empleado.findOne({ Clave: datos.Clave });
        if (existente) return res.status(409).json({ error: `La Clave ${datos.Clave} ya está registrada.`
});
        const nuevo = new Empleado(datos);
        const guardado = await nuevo.save();
        res.status(201).json({ mensaje: 'Empleado registrado exitosamente', empleado: guardado });
    } catch (error) {
        res.status(400).json({ error: 'Error al crear empleado', mensaje: error.message });
    }
});

// PUT: Actualizar empleado por Clave o _id
app.put('/api/empleados/:identificador', requireDB, async (req, res) => {
    try {
        const { identificador } = req.params;
        const filtro = !isNaN(identificador) ? { Clave: Number(identificador) } : { _id: identificador };
        const actualizados = normalizarDatosEmpleado(req.body);
        const empleado = await Empleado.findOneAndUpdate(filtro, { $set: actualizados }, {
returnDocument: 'after', runValidators: true });
        if (!empleado) return res.status(404).json({ error: 'Empleado no encontrado' });
        res.status(200).json({ mensaje: 'Empleado actualizado exitosamente', empleado });
    } catch (error) {
        res.status(400).json({ error: error.message });
    }
});

// DELETE: Eliminar empleado por Clave o _id
app.delete('/api/empleados/:identificador', requireDB, async (req, res) => {
    try {
```

```
        const { identificador } = req.params;
        const filtro = !isNaN(identificador) ? { Clave: Number(identificador) } : { _id: identificador };
        const eliminado = await Empleado.findOneAndDelete(filtro);
        if (!eliminado) return res.status(404).json({ error: 'Empleado no encontrado' });
        res.status(200).json({ mensaje: 'Empleado eliminado exitosamente', empleado: eliminado });
    } catch (error) {
        res.status(500).json({ error: error.message });
    }
});

const PORT = process.env.PORT || 3000;
if (process.env.NODE_ENV !== 'production' || require.main === module) {
    app.listen(PORT, () => console.log(`Servidor local en puerto ${PORT}`));
}
module.exports = app;
```

ARCHIVO: VERCEL.JSON

```
{
  "version": 2,
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/api/index.js"
    }
  ]
}
```

ARCHIVO: API/INDEX.JS

```
const app = require('../index');

module.exports = app;
```